BrainyKids

Security implementation review & action plan

16 October 2026 / Source and evidence review

24 implementation actions	**12** pre-launch P0 gates	**10** isolated evidence probes

Launch decision
Close the P0 trust-boundary, privacy and business-integrity gaps before public multi-family production use.

Prepared against commit **9452412** using the supplied five-check security guide, repository source, the existing behavioural report and local validation.

Documentation-only assessment. No application fixes or production penetration tests are represented as completed. No real credentials or child identifiers are reproduced here.

Reading guide: scope and report comparison; five-check scorecard; data-flow map; 24 prioritized findings with evidence, reason, impact, fix and acceptance criteria; validation and release gates. PDF bookmarks provide section and finding navigation.

1. Decision and scope

Review date: 16 October 2026 (UTC). **Reviewed baseline:** `9452412918ac7979458fbc26f2f4521a47791598`.
Repository: `Raheelatta1984/BrainyKids` (public at review time).

Recommendation: do not treat the current implementation as ready for a public, multi-family production launch until the P0 items below are remediated and independently retested. This is a review and implementation backlog, not a security certification. No application vulnerabilities were fixed by this documentation PR.

The review applies the five checks in the supplied *5 Security Checks Before You Launch Your App*, Mayank Shah, free resource v1.1, pages 3-7: secrets, personal data flows, production readiness, complex authentication/business logic, and attacker-perspective abuse. The original user-supplied PDF is preserved in `docs/security/reference/vibe-coding-security-prompts.pdf`; attribution remains with its author. SHA-256: `c4b5434f8b3ed27590a4ac584092a6c21e0aee358ebff34d31305892545acc90`.

Reviewed: all API route files, authentication/session/passkey/pairing helpers, storage and reward logic, client voice/data handling, dependency manifest and lockfile, deployment examples, README, and the existing behavioural report and inbox logs. Ten isolated evidence probes were run using dummy data; no live accounts or production data were attacked. No deployment credentials were needed.

Limits: no production URL, proxy configuration, database, cloud console, backup system or legal policy was supplied. These operational controls remain unverified. The local Git clone is shallow (one baseline commit), so historical secret exposure was not exhaustively assessed. Gitleaks, Bearer, ECC and Trail of Bits tools were not executed; the attachment's categories were used as a manual review framework, not as evidence of those tools having passed. No external penetration test or browser exploit chain was performed.

Priority definitions: P0 = before public production; P1 = next hardening milestone; P2 = planned operational assurance. Severity expresses plausible confidentiality, integrity or availability impact, not a CVSS score. A conditional finding requires the stated deployment condition. Every item remains OPEN; listed owners are proposed roles, not assigned people.

2. Review of the existing report

`logs/behaviour-log.md` was generated on 14 October 2026 and covers approximately five minutes. Its summary contains 7 error, 5 warning and 9 informational events. It is a behavioural/reliability report, not a security audit or evidence of a breach.

Its central conclusion is sound: non-durable serverless storage can lose accounts, children and device links. However, its storage header describes a permanent file while individual events also describe temporary storage; it aggregates multiple environments. Neither that header nor this source review proves that production currently has a durable database.

What the report misses: the authentication impact of restoring identity from client cookies; predictable signing-key fallback; readable credential hashes on child devices; cross-instance revocation and challenge replay; login/PIN brute force; permissive passkey origin policy; diagnostic isolation/abuse; exposed quiz answers; client-trusted timing; ledger retention undermining earning caps; privacy deletion and secure deployment evidence.

Change to its advice: attaching a database is necessary for hosted multi-instance use, but insufficient alone. Remove identity reconstruction from client backups, require an independent signing secret, keep hashes exclusively server-side, and test revocation across instances. An auto-restored session is not automatically a secure session.

The tracked report and JSONL files contain child display names, stable child/device identifiers, pairing codes, timestamps and internal filesystem paths. Their real-versus-synthetic status is unknown. Treat them as potentially personal data; this new report deliberately does not repeat those names or identifiers.

3. Five-check scorecard

- **01 / Secret leak prevention: FAIL / partial controls.** Environment files are ignored except the placeholder example; no matches in a targeted credential-pattern scan. A predictable signing fallback and client-cookie hash exposure remain. Full Git-history scanning and rotation verification are outstanding.
- **02 / Personal data flows: FAIL.** Salted scrypt hashing and normal-response view filters exist, but credential hashes travel in backup cookies; deletion is incomplete; logs and shared diagnostics expose more than necessary.
- **03 / Production audit: FAIL in application; deployment partly UNKNOWN.** Signup throttling and generic unexpected-error responses exist. Critical configuration is optional; security headers and general abuse controls are absent from source; database TLS peer verification is not enforced.
- **04 / Complex logic audit: FAIL / partial controls.** Parent ownership checks and server-calculated awards exist, but trust-boundary weaknesses, answer disclosure, timer bypass and retention-based cap bypass remain.
- **05 / Attacker perspective: FAIL / testing incomplete.** Ten local probes confirm unsafe behaviour. Cross-family HTTP authorization, browser CSRF/XSS and production exposure still need end-to-end testing after fixes.

Not applicable in this version: no payment processor, card collection, payment webhook, OAuth integration, Supabase client/RLS integration, file-upload endpoint or external AI API was found. Rewards are an internal ledger paid outside the app, not a payment gateway. Do not implement Stripe/webhook controls unless those features are introduced. PostgreSQL tenant isolation is nevertheless essential. Stable internal IDs are not secrets by themselves: enforce ownership rather than hiding IDs as an authorization control.

4. Personal-data flow map

- **Parent identity and credentials:** signup/login/recovery collect name, email, password and recovery code. Hashes go to server JSON or the shared Postgres document; parent views return identity fields. Backup cookies additionally contain parent records with password/recovery hashes, including on child devices. Required: opaque cookies, server-only hashes, deletion and retention.
- **Child identity and learning:** parent forms collect name, username, age, optional gender/theme and curriculum; PIN is hashed. Profiles, progress, questions and activity live in the server document; snapshots and SSE send authorized family/child views. Backup cookies also include the child record/PIN hash. Required: minimize data and role-specific responses; never serialize the full record into cookies.
- **Chat and voice:** typed and recognized speech text goes to family chat APIs and the document. No app-owned audio upload/store or analytics/AI request was found. Browser SpeechRecognition and speech synthesis may use browser/OS/vendor services; source alone cannot prove on-device processing. Required: clear guardian disclosure, optional voice, provider/region assessment and deletion of transcripts.
- **Devices, sessions and pairing:** random bearer tokens are hashed server-side, except temporary pairing handoff tokens and backup token copies. User-agent-derived labels, codes and status enter document/events/logs. HttpOnly SameSite=Lax cookies carry bearer state. Required: expiry, revocation, no secret-bearing diagnostic data and bounded handoff lifetime.
- **Network and diagnostics:** signup consumes forwarded IP headers and persists a keyed HMAC, not a raw IP. Client diagnostic payloads include URLs, device context, names/IDs and arbitrary bounded strings. Server logs and exports duplicate some data into JSONL and Git. Required: trusted proxy policy, field allowlists, redaction, strict family scope, quotas and retention.
- **Rewards and settings:** question outcomes, balances, earning schedule, currency and parent approvals persist in the document/ledger and appear in family views/events. No bank/card/payment credentials are collected. Required: immutable accounting, durable earning aggregates and concurrency tests.
- **External infrastructure:** with database configuration, the server sends the complete application document to configured Postgres hosting. Hosting/proxy logs and backups may hold further data; actual providers, access policies, encryption, residency and retention are not verified. Client application fetches use relative same-origin API paths.

5. Prioritized implementation register

SEC-01 | P0 | Critical (conditional) | Require an independent signing secret

Status/evidence: Confirmed default-key behaviour; exploitation depends on absent effective secret/database seed and use of continuity paths. `src/lib/server/auth.ts:31-38` derives the key from optional variables or a public constant. Probe 1 verifies acceptance of an externally signed dummy payload.

Reason and impact: a public value is not a secret. In temporary-storage mode forged continuity payloads can reconstruct attacker-chosen identity records, enabling authentication/role compromise. Database-derived keys also couple database credential rotation to unrelated cookie security.

Implement: require a cryptographically random, independently managed secret of at least 32 bytes in production; validate startup configuration; remove public/database fallbacks; separate keys by purpose, define key versions and controlled rotation. Invalidate old continuity cookies when migrating.

Acceptance / owner: production refuses missing/weak keys; dummy public-key signatures are rejected; rotation invalidates old identities consistently across instances. Owner: backend + platform.

SEC-02 | P0 | High | Keep credential hashes out of browser cookies

Status/evidence: Confirmed. `src/lib/server/session.ts:24-39, 110-142` serializes the full parent and child records; `slimParent` retains password/recovery hashes. Probe 4 decodes a child backup cookie and finds all three credential hashes.

Reason and impact: HMAC signing provides integrity, not encryption. A device owner, cookie theft or export can reveal parent hashes and child PIN hashes for offline guessing. HttpOnly prevents JavaScript access, not access by the device owner. This crosses the parent/child trust boundary even with a strong signing key and durable storage.

Implement: remove backup identity records and use opaque random session IDs backed by durable storage. Never send password, PIN or recovery hashes to any client. Inventory affected cookie lifetimes and require credential/PIN reset where exposure is suspected; rotate sessions during migration.

Acceptance / owner: decoded cookies and all role-specific responses contain no credential hashes, full identity records or backup bearer tokens. Owner: backend/security.

SEC-03 | P0 | High | Fail closed without durable production state

Status/evidence: Confirmed permissive code path; actual deployment unknown. `src/lib/server/store.ts` (`getStore`, `fileDriver`) permits local/temporary JSON; `.env.example` treats database and secret as optional. Existing report records lost data.

Reason and impact: restart or instance changes can erase identities, revocations, throttles and reward state. Continuing in degraded mode creates both data loss and inconsistent access decisions.

Implement: require durable shared storage for hosted production, readiness checks and deployment gates. Restrict file mode to explicit local/single-instance development. Return safe 503s instead of accepting production identity operations with temporary state. Backfill and verify existing data before rollout.

Acceptance / owner: missing/unavailable database prevents production readiness and auth writes; two instances observe identical state after restart. Owner: platform/backend.

SEC-04 | P0 | High | Make revocation and challenge consumption authoritative

Status/evidence: Confirmed device resurrection in non-durable mode (probe 6). `session.ts:47-88, 150-190`, `service.ts` (`unlinkDevice`), and `passkeys.ts` (`takeChallenge`) allow signed recovery when records disappear; consumed fallback challenges are remembered only in a process-local Set. Deletion tombstones in `store.ts` are capped at 200.

Reason and impact: a copied cookie can regain access after unlinking on temporary storage. Another instance or process restart cannot reliably know a challenge was consumed or an identity deleted. Parent restoration can also reintroduce stale identity state after storage loss.

Implement: delete identity restoration fallback; store revocation, expiry, session versions and single-use challenges durably and atomically. Do not evict security tombstones while related credentials remain valid. Force fresh authentication rather than reconstructing accounts.

Acceptance / owner: replaying old device/parent cookies after logout, reset, unlink or deletion fails across two instances and restarts; the same challenge succeeds at most once globally. Owner: backend.

SEC-05 | P0 | High | Add comprehensive authentication throttling

Status/evidence: Confirmed absent limiter in parent login, child login and parent-password child logout routes. Recovery has five-failure account lockout but no per-client/global quota; signup alone has bounded limits.

Reason and impact: 4-6 digit PINs are cheaply enumerable online. Password spraying, recovery lockout abuse and synchronous scrypt work under a shared mutation lock threaten all families' availability.

Implement: shared atomic per-IP, per-account and global limits on login, kid PIN, recovery, password checks and passkey options/verify; progressive delay, 429/Retry-After, bounded queues and edge protection. Start with the guide's 5 login attempts/minute and 3 recovery attempts/hour per client, then tune for family/shared-network usage. Sanitize trusted proxy headers; do not rely only on an attacker-supplied IP.

Acceptance / owner: parallel and distributed attempts are limited across instances without revealing account existence; legitimate shared networks have a recovery path. Owner: backend/platform.

SEC-06 | P0 | High | Use a strict WebAuthn origin and RP policy

Status/evidence: Confirmed. `src/lib/server/webauthn.ts:428-450` adds the request Origin and HTTP/HTTPS host even when `BRAINYKIDS_ORIGINS` is configured. Probe 2 shows an untrusted origin entering the allowlist.

Reason and impact: request-controlled values cannot establish the authentication trust boundary. This weakens origin isolation, especially for attacker-controlled sibling origins under a configured RP domain. Signature/RP checks still apply; this is not evidence that any arbitrary site can directly steal a passkey.

Implement: use validated canonical HTTPS origins and RP ID in production, reject unknown Origin/Host values, trust forwarding headers only from the edge, and allow local HTTP only in explicit development. Prefer a maintained WebAuthn library with equivalent tests over a custom protocol parser.

Acceptance / owner: trusted-origin signatures pass; sibling/unconfigured origin, spoofed host, cross-origin and insecure production requests fail even if their cryptographic assertions are otherwise valid. Owner: backend/security.

SEC-07 | P0 | High (defence gap) | Enforce CSRF protection on mutations

Status/evidence: Source-confirmed absence of a centralized Origin, Fetch Metadata or CSRF-token guard in `src/lib/server/http.ts` and mutation handlers. `readBody` parses JSON without enforcing content type. Cookies use SameSite=Lax.

Reason and impact: Lax is useful but not sufficient for every same-site sibling-origin scenario; text/plain requests can reach JSON parsing. Cross-origin login/session manipulation and privileged writes need an explicit policy. Full browser exploitability depends on origin topology and was not demonstrated.

Implement: reject untrusted origins on all cookie-authenticated mutation routes, enforce application/json with bounded parsing, use Fetch Metadata and/or bound CSRF tokens, and protect login/logout flows. Keep CORS restrictive; CORS alone does not prevent requests.

Acceptance / owner: browser tests cover unrelated sites, sibling subdomains, null/missing origins, text/plain JSON and valid same-origin clients. Unauthorized requests make no state changes. Owner: backend/frontend.

SEC-08 | P0 | High | Do not disclose active question answers

Status/evidence: Confirmed in probe 8. `src/app/api/kid-session/questions/next/route.ts` returns `issueNextQuestion` directly; `src/lib/question-engine.ts:318-345` includes `answer`, `explain` and template values that may contain the answer.

Reason and impact: a linked child device can read the answer before submitting and automate correct responses for points and money/Robux ledger rewards. Server-side scoring is insufficient if the scoring secret is sent first.

Implement: define an allowlisted public question DTO containing only display fields and choices; withhold canonical answers, explanations and answer-bearing translation metadata until after an accepted submission. Translate display text server-side or use safe template parameters. Retain authoritative question state server-side.

Acceptance / owner: next-question JSON contains no answer or derivable hidden answer metadata; answer explanations arrive only after submission; server rejects forged/replayed question IDs. Owner: backend/frontend.

SEC-09 | P1 | Medium | Measure question deadlines on the server

Status/evidence: Confirmed in probe 9. `src/lib/server/service.ts:130-184` stores no issuance timestamp/deadline and decides timeout using client `elapsedSeconds`; the answers route defaults it to zero.

Reason and impact: a client can pause indefinitely or report zero/negative time and still earn rewards. This violates timer and fairness rules, although rewards are paid externally by parents.

Implement: persist issuedAt/expiresAt with each question, evaluate server time atomically on submission and treat client elapsed time as telemetry only. Define reconnection/grace behaviour explicitly.

Acceptance / owner: overdue submissions fail even with zero, negative, omitted or malformed elapsed values; accepted retries do not double-credit. Owner: backend.

SEC-10 | P1 | Medium | Enforce device expiry server-side

Status/evidence: Confirmed in probe 5. `session.ts:150-159` accepts any matching device record; `service.ts` (`linkDevice`) has no expiresAt. The 180-day setting in `constants.ts` is only a browser-cookie lifetime.

Reason and impact: a copied token supplied manually can outlive browser expiry. Password changes/reset revoke parent sessions but do not establish a clear policy for child-device tokens and passkeys.

Implement: store and check absolute/idle expiry, revocation and authentication version for devices; rotate tokens appropriately. Provide parent session/device management and an explicit revoke-all option covering compromised child links/passkeys as policy requires.

Acceptance / owner: a manually replayed expired device token fails; reset/revoke-all tests cover parent sessions, child links, passkeys and cached snapshots/SSE. Owner: backend/product.

SEC-11 | P0 | High | Isolate diagnostic data by family

Status/evidence: Confirmed in probe 7. `src/lib/server/diagnostics.ts:274-281` excludes other non-null parent IDs but includes unattributed entries for every parent. `/api/diagnostics` POST accepts guest events, arbitrary text fields and client-supplied child/device identifiers; GET exposes filtered entries to parents.

Reason and impact: unattributed logs can leak internal errors or visitor-provided personal data across families. Client-supplied identity labels can make reports misleading. This is a concrete exception to otherwise useful ownership checks.

Implement: require exact parentId equality for family logs; place sanitized deployment health in a separate non-personal channel. Derive role/IDs from authentication, validate referenced child ownership and prohibit guests from setting identity fields. Use per-event field allowlists.

Acceptance / owner: two-family plus guest tests prove no cross-family/unattributed personal event appears in JSON, Markdown, JSONL or SSE; spoofed IDs are ignored/rejected. Owner: backend/privacy.

SEC-12 | P1 | High | Bound abuse and public metadata exposure

Status/evidence: Confirmed open surfaces; edge controls unknown. `/api/diagnostics` accepts guest writes without a rate limit; `/api/pairing/start` allocates records; `/api/events` opens polling streams. `/api/storage` and guest `/api/snapshot` expose storage metadata; storage errors can name host/path/environment variables. Pairing lookup returns non-pending status before its ownership check.

Reason and impact: callers can churn logs, allocate storage/streams and exhaust the single shared document lock. Metadata assists reconnaissance; code probing can reveal pairing state. Existing length/count caps do not bound request frequency or concurrent work.

Implement: quotas per client/family/device and globally, SSE connection caps, pairing collision checks and attempt limits, minimal public health responses, protected detailed diagnostics, ownership-first pairing lookup and sanitized errors with correlation IDs. Verify trusted edge IP handling.

Acceptance / owner: load/abuse tests stay within CPU/memory/database budgets; unauthenticated callers receive no database host, filesystem path or family-specific status. Owner: platform/backend.

SEC-13 | P1 | High | Complete deletion and child privacy controls

Status/evidence: Confirmed incomplete deletion. Child DELETE in `src/app/api/kids/[id]/route.ts:121-170` removes profiles/devices/pairings/chats but not all passkeys, challenges, rewards, events or behavioural/file/Git copies. No parent-account deletion route was found.

Reason and impact: families cannot reliably remove retained personal data. Orphaned credentials and archives complicate access revocation and privacy obligations. This review does not determine which laws apply.

Implement: authenticated, reauthenticated account/family deletion; cascade revoke and delete/anonymize all linked records, log exports and cookies; documented backup expiration and audit exceptions. Publish data categories, guardian controls, retention, voice processing and processor disclosures. Obtain legal advice for intended jurisdictions and child-user age ranges.

Acceptance / owner: seeded family deletion leaves no usable credentials or identifiable data outside documented lawful retention; deleted users cannot return via backup cookies; retention/backup jobs are demonstrated. Owner: backend/product/privacy.

SEC-14 | P0 | High (potential exposure) | Stop committing personal diagnostic data

Status/evidence: Confirmed content in tracked `logs/behaviour-log.md`, `logs/behaviour-summary.json` and `logs/inbox/*.jsonl`; whether names represent real children is unverified. The repository is public. `http.ts:56-74` logs full request URLs/raw causes and persists exception messages; diagnostics accepts arbitrary text without secret-aware redaction.

Reason and impact: names, stable identifiers, pairing codes, device details, URLs and internal paths can be distributed permanently through Git and exports. Arbitrary error/context strings may contain secrets; no live credential leak was identified by this review.

Implement: immediately verify whether existing fixtures are synthetic. Quarantine real diagnostics outside Git, commit only irreversibly anonymized fixtures, and add a review/redaction gate. If actual personal data or secrets were exposed, initiate incident assessment, revoke/rotate affected credentials and coordinate any approved history cleanup; deleting a file does not erase Git copies. Use structured allowlisted logging with retention and access control.

Acceptance / owner: secret/PII scanners and manual fixture review pass; logs strip query tokens and sensitive fields; incident decision and any cleanup are recorded privately. Owner: security/privacy + maintainers.

SEC-15 | P1 | Medium | Add tested production security headers

Status/evidence: No CSP, frame-ancestors, X-Frame-Options, HSTS, nosniff, Referrer-Policy or Permissions-Policy configuration found in `next.config.ts` or a global response layer. CDN-injected headers were not checked.

Reason and impact: missing hardening increases script-injection, clickjacking, content-sniffing and transport-downgrade exposure. React text escaping is helpful but not a complete XSS policy.

Implement: application-wide nosniff, frame-ancestors 'none'/X-Frame-Options DENY, suitable Referrer-Policy and Permissions-Policy, production HTTPS redirects and HSTS (one year after validating HTTPS coverage). Introduce nonce/hash-based Next.js-compatible CSP, report-only first, then enforce. Explicitly scope microphone and WebAuthn permissions; separate development/embedded preview needs from production.

Acceptance / owner: response tests cover pages, JSON/errors, streams and proxy responses; no broad unsafe script policy; voice and passkeys still work in supported production contexts. Owner: frontend/platform.

SEC-16 | P0 | High (configuration risk) | Verify database TLS peers

Status/evidence: Code-confirmed policy gap; actual connection unknown. `src/lib/server/store.ts:385-450` normalizes known cloud hosts to sslmode=require with uselibpqcompat=true and accepts disable/prefer or arbitrary hosts. Under libpq-compatible semantics, encryption-only require is not equivalent to hostname/certificate verification.

Reason and impact: encrypted traffic without authenticated peers can permit database interception; permissive connection modes may permit plaintext. Source cannot confirm production network ACLs or certificates.

Implement: require verified TLS/verify-full or explicit trusted CA plus hostname validation in production. Reject insecure modes except explicit local development; document pooler-specific trust configuration. Confirm private/allowlisted network access, non-default credentials and encrypted storage/backups with the provider.

Acceptance / owner: real database integration tests reject untrusted CA, hostname mismatch and plaintext fallback, while the configured trusted endpoint succeeds. Owner: platform/database.

SEC-17 | P1 | Medium (toolchain) | Remediate dependency advisories deliberately

Status/evidence: `npm audit --json` on 16 October 2026 reports 9 affected package entries: 5 high and 4 moderate. `npm audit --omit=dev --json` reports 0. Raw snapshots are saved alongside this report. Multiple entries are transitive effects, not nine independent exploitable flaws.

Reason and impact: high entries are in the eslint-config-next -> fast-glob -> micromatch -> braces chain (stack-exhaustion advisory GHSA-vfj7-8cjw-p6xm). Moderate entries include drizzle-kit/esbuild-kit/esbuild; esbuild advisories include development-server cross-origin access (GHSA-67mh-4wv8-2f99) and a Windows local file-read advisory (GHSA-g7r4-m6w7-qqqr). Exposure depends on tool usage, OS and untrusted inputs; no production exploit was demonstrated.

Implement: evaluate supported compatible upgrades/removal of unused tools or narrowly justified overrides; rerun lockfile audit and all builds/tests. Do not blindly run npm audit fix --force: reported fixes propose major-version changes/downgrades. Do not expose development tooling to untrusted traffic.

Acceptance / owner: no unexplained high findings; documented owner/expiry for exceptions; reproducible npm ci and production/toolchain audits in CI. Owner: maintainers.

SEC-18 | P1 | Medium | Validate bounded input before any mutation

Status/evidence: Probe 3 confirms `readBody` reads the entire request then returns an empty object above 64,000 characters; it also accepts arrays and ignores content type. Signup/child names lack consistent maximums. Child PATCH changes fields in the live document before later validation can return an error, and `store.mutate` still persists its callback result.

Reason and impact: large bodies consume memory before rejection, malformed values can trigger unintended defaults, and rejected updates can partially persist. Long record data also worsens cookie/document growth.

Implement: byte-limited streaming parsing plus edge caps, 413/415/400 distinctions, strict object schemas, bounded strings/numbers and allowlisted enums. Validate a candidate copy fully before applying it atomically; invalid callbacks must not commit changes.

Acceptance / owner: oversized/multibyte bodies, arrays, wrong content types and invalid mixed PATCHes leave state unchanged; use fuzz/property tests. Owner: backend.

SEC-19 | P1 | Medium | Require recent authentication for sensitive actions

Status/evidence: recovery-code issuance, passkey registration/deletion and child credential updates rely on any valid session; parent sessions last 30 days. Password changes correctly require the current password, but current-password verification is outside the mutation and the same session token is retained.

Reason and impact: a stolen/unattended session can establish persistent account recovery or passkey access. Concurrent credential changes may use stale authentication checks.

Implement: password or verified passkey step-up with short recent-auth TTL for recovery rotation, credential changes and deletion. Recheck credential version transactionally, rotate session tokens after step-up/password changes, and offer safe notifications/session review without exposing new recovery secrets.

Acceptance / owner: stale sessions cannot add/reset recovery credentials; competing password changes cannot both succeed against stale state; old tokens are invalidated. Owner: backend/product.

SEC-20 | P1 | Medium | Harden passkey verification and discovery privacy

Status/evidence: registration/login use userVerification='preferred'; `webauthn.ts:352,400` checks user presence but not the UV bit. Public login/options returns all credential IDs/transports; status returns parent/child counts. No attestation-chain validation is implemented, as README acknowledges.

Reason and impact: presence is not proof of biometric/PIN verification, despite fingerprint/face-unlock wording. Global credential discovery leaks deployment metadata and grows with every family. Attestation 'none' is not itself a vulnerability and hardware attestation is not required for ordinary consumer passkeys.

Implement: decide/communicate assurance policy; if verified local unlock is required, request and enforce UV. Use discoverable credentials without global allowCredentials enumeration or an account-scoped non-enumerating flow. Add maintained-library negative tests for flags, algorithms, CBOR limits, crossOrigin and replay.

Acceptance / owner: UV-absent responses fail when required; anonymous responses disclose no global credential list/counts; supported authenticators still work. Owner: backend/security.

SEC-21 | P1 | Medium | Reduce shared-state blast radius

Status/evidence: `store.ts` stores all families in one brainykids_state JSON row, creates its table at runtime and locks the row for mutations. Chat and other history caps are global, despite some family-oriented comments. Actual database role grants are unknown.

Reason and impact: one tenant's traffic contends with everyone; a server/query mistake has broad data access, and global truncation lets one family displace another's history. In-process file serialization is not cross-process isolation.

Implement: separate tenant-keyed tables or explicit family partitions, constraints, transactional accounting and per-family quotas; use DB RLS only where appropriate to the access model. Separate migration credentials from a least-privilege runtime role; verify grants and encrypted backups. Do not assume RLS alone fixes an application-wide service identity.

Acceptance / owner: two-family isolation/load tests, role-grant review and schema constraints prevent cross-family reads/writes and history eviction; runtime cannot perform schema administration. Owner: backend/database.

SEC-22 | P2 | Medium (unverified operations) | Prove recovery and incident readiness

Status/evidence: no production backup/restore evidence, incident runbook or recovery-objective record was supplied or found in the reviewed files. This is not proof that the hosting provider lacks these controls.

Reason and impact: storage loss, stolen credentials or privacy incidents can become prolonged outages or irreversible loss without tested recovery and response ownership. Client cookies are not a backup strategy.

Implement: define RPO/RTO, encrypted backups with restricted restore access, scheduled restore drills, secret/session revocation and compromised-key runbooks, security alerts and incident contacts. Monitor failed auth, abuse rates, durable-state readiness and ledger reconciliation without logging personal content.

Acceptance / owner: restore a sanitized backup into an isolated environment and record achieved RPO/RTO; tabletop-test key compromise and child-data exposure; evidence and on-call ownership reviewed periodically. Owner: platform/security.

SEC-23 | P1 | Medium | Make security checks mandatory in CI and GitHub

Status/evidence: no tracked `.github` workflow or SECURITY.md was found. Existing selftests are valuable but do not cover HTTP tenant boundaries/CSRF, secret history, deadlines or distributed revocation. GitHub branch protection, required checks, secret scanning and access policy were not inspected.

Reason and impact: functional tests can pass while exploitable trust-boundary regressions ship. Unreviewed dependency/source changes can bypass security review.

Implement: CI for npm ci, lint, typecheck, build, all selftests, security regressions, production/dev dependency audit, secret scanning (including full history initially), and static data-flow analysis. Configure protected main, required reviews/checks, minimal Actions permissions, pinned trusted actions and a private vulnerability-reporting path. Require MFA and least-privilege maintainer access. Rotate any historically exposed real secret; history cleanup alone is insufficient.

Acceptance / owner: a test PR containing a safe synthetic secret or broken ownership/expiry check is blocked; maintainers verify actual GitHub settings and required status checks. Owner: maintainers/security.

SEC-24 | P0 | High (business integrity) | Make earning caps independent of truncated history

Status/evidence: Confirmed in probe 10. `store.ts:1019-1026` truncates the reward ledger globally at 2,000 entries; `views.ts` (`earnedSums`) computes daily/weekly totals only from that retained ledger. After other-family entries evict an earned transaction, the first family's daily/weekly earned totals drop to zero.

Reason and impact: enough activity can erase evidence used to enforce earning limits, enabling additional reward credit beyond intended caps and losing accounting history. This is an internal ledger integrity risk, not a demonstrated theft from a payment gateway. Separately, pending payout values need a defined policy when balances or reward currency change before approval.

Implement: maintain durable transactionally updated per-child/day/week earning aggregates independent of display/history retention; preserve auditable accounting with family-scoped retention. Use immutable request IDs, snapshotted reward unit/currency and well-defined balance reservation/reconciliation rules for approval; retain duplicate-answer protections and parent ownership checks.

Acceptance / owner: more than 2,000 cross-family transactions never reset another child's limits; concurrent answers/approvals and duplicate requests cannot double-credit/pay, produce invalid balances or mix currencies. Owner: backend/product.

6. Existing controls worth retaining

- Salted scrypt for passwords, PINs and recovery codes; random 32-byte session tokens; server-side token hashes and timing-safe comparisons (`auth.ts`). SHA-256 for high-entropy tokens is appropriate; it is not used as the password hash.
- HttpOnly, SameSite=Lax cookie flags; Secure in production defaults and HTTPS-aware device flows. However forwarded protocol trust and production HTTP must be constrained; browser flags do not replace server expiry or CSRF protection.
- Explicit parentView/kidView normal-response filtering; route-level parent/device checks; parent ownership constraints on child/device/chat/reward mutations. No broad ownership bypass was observed in those inspected routes, but this is not an end-to-end IDOR pass.
- Postgres parameterized document writes, transaction/row-lock serialization and generic unexpected-error responses. There is no obvious user-input string interpolation into the reviewed SQL.
- Server-side correctness and reward calculations, issued-question matching, duplicate-answer rejection, one-time recovery-code rotation and invalidated parent sessions on password recovery.
- Signup burst/daily caps, HMAC IP pseudonyms and bounded counter state; WebAuthn signature, challenge, RP hash, origin membership and counter checks. Harden their policy and shared-state semantics rather than discarding all existing tests.
- No dangerous HTML rendering or localStorage-based personal-data persistence was found in the inspected client source. No external analytics/payment/AI integration was found; verify future additions separately.

7. Validation record and evidence

All successful on 16 October 2026: npm ci --ignore-scripts; npm run lint; npm run typecheck; npm run build; npm run selftest (31 WebAuthn/storage checks); npm run levels-selftest; npm run quiz-selftest; npm run signup-selftest. These functional passes do not establish security readiness. npm audit exits nonzero for its findings; the production-only audit reports zero.

Targeted secret-pattern check: 107 baseline tracked files scanned for private-key headers, GitHub tokens, AWS access IDs, Stripe live secrets and OpenAI-style keys; zero pattern matches. Only `.env.example` is tracked as an environment file. This narrow check is not an entropy scan, not full-history scanning, and does not override SEC-01/02/14.

Evidence probes: run `npx tsx docs/security/review-probes.ts`. Ten checks confirm: public-key signature acceptance under fallback configuration; request-widened passkey origins; oversized body silently becoming an empty object; readable credential hashes in backup cookies; missing device expiry; removed-device restoration; unattributed diagnostic scope; pre-submission answer disclosure; client-controlled timeout; and global ledger eviction erasing earned totals. All use fixtures/in-memory state, no production requests.

Important: these probes deliberately PASS when reviewed unsafe behaviour exists. They are dated evidence tools, not security regression gates. Replace them with rejection/invariant assertions during remediation; a future probe failure may mean a finding was fixed. Baseline probe numbers map to SEC-01, 06, 18, 02, 10, 04, 11, 08, 09 and 24 respectively.

Artifacts: report Markdown/PDF, the evidence-probe TypeScript file, both npm audit JSON snapshots, the supplied reference PDF, and a reproducible PDF renderer/requirements in this directory. Snapshot advisory URLs are supporting dependency evidence, not a claim that every toolchain finding is reachable in production.

8. Delivery sequence and release gates

- **Contain immediately:** determine whether tracked diagnostics are real, restrict public exposure if needed, provision an independent secret and durable storage, revoke legacy continuity state, and prohibit production reliance on identity backups. Record incident decisions privately.
- **Before public launch (P0):** close SEC-01 through SEC-08, SEC-11, SEC-14, SEC-16 and SEC-24. For conditional issues provide verified configuration evidence and tests, not only documentation. Review any accepted residual risks with a named accountable owner.
- **Next hardening milestone (P1):** complete expiry/deadline enforcement, quotas, privacy/deletion, headers, dependencies, validation, step-up, passkey privacy, tenant isolation and CI. Include concurrent/two-instance integration tests and role-specific response snapshots.
- **Operational assurance (P2):** demonstrate restoration and incident exercises; engage an independent human reviewer for child-data handling and production launch.

Release checklist: every P0 has an owner, fix commit and negative-test evidence; two families cannot cross-read/write; guest logs cannot leak; old sessions/challenges cannot revive; no hashes/active answers leave server trust; rewards respect immutable caps; production verifies TLS and canonical origins; dependency exceptions are approved; backup, privacy and incident ownership is recorded. Keep evidence free of real credentials and children's data.

9. Repository delivery notes

This change adds review artifacts only; it does not silently apply the source attachment's instructions to change application behaviour. The assessment is pinned to the baseline, so subsequent fixes should reference finding IDs and update a separate remediation tracker rather than rewriting historical evidence.

The requested PR is to be opened from the session branch `arena/bb87c5a3-brainykids` into `main`, then merged explicitly after checks/review. The session cannot switch to or delete other branches. Branch cleanup is therefore excluded; the session branch must remain. At initial remote inspection only main existed; publishing this PR adds the session branch. Do not represent this review as implementing the backlog or completing branch cleanup.